

COMP90038 Algorithms and Complexity

Analysis of Algorithms

Junhao Gan

Week 2 Lecture 4

Semester 1, 2026

Establishing Growth Rate

We just showed $f(n) \in O(g(n))$ for some cases of f and g , using the definition of big- O notation directly:

$$n > c_2 \Rightarrow f(n) < c_1 \cdot g(n)$$

for some constants c_1 and c_2 . A more common approach uses

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \begin{cases} 0 & \text{implies } f \text{ grows asymptotically slower than } g \\ c_1 & \text{implies } f \text{ and } g \text{ have same order of growth} \\ \infty & \text{implies } f \text{ grows asymptotically faster than } g \end{cases}$$

Use this to show that $1000n \in O(n^2)$.



L'Hôpital's Rule

It is often helpful to use L'Hôpital's rule:

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \lim_{n \rightarrow \infty} \frac{f'(n)}{g'(n)}$$

where f' and g' are the **derivatives** of f and g .

For example, we can show that $\log_2 n$ grows slower than $\sqrt{n}$:

$$\lim_{n \rightarrow \infty} \frac{\log_2 n}{\sqrt{n}} = \lim_{n \rightarrow \infty} \frac{(\log_2 e) \frac{1}{n}}{\frac{1}{2\sqrt{n}}} = 2 \log_2 e \lim_{n \rightarrow \infty} \frac{1}{\sqrt{n}} = 0$$

Example: Finding the Largest Element in an Array

function MAXELEMENT($A[\cdot], n$)

▷ Array A and its size n , index starting from 0

$max \leftarrow A[0]$

for $i \leftarrow 1$ to $n - 1$ **do**

if $A[i] > max$ **then**

$max \leftarrow A[i]$

return max

We count the **number of comparisons** executed for an array of size n :

$$C(n) = \sum_{i=1}^{n-1} 1 = n - 1 \in O(n)$$

Think: How could we find the smallest element in an array?

Example: Selection Sort

```
function SELSORT( $A[\cdot]$ ,  $n$ )  
  for  $i \leftarrow 0$  to  $n - 2$  do  
     $min \leftarrow i$   
    for  $j \leftarrow i + 1$  to  $n - 1$  do  
      if  $A[j] < A[min]$  then  
         $min \leftarrow j$   
    swap  $A[i]$  and  $A[min]$ 
```

▷ Shorthand for 3 assignments

We count the **number of comparisons** executed for an array of size n :

$$\begin{aligned} C(n) &= \sum_{i=0}^{n-2} \sum_{j=i+1}^{n-1} 1 = \sum_{i=0}^{n-2} (n-1-i) = (n-1)^2 - \sum_{i=0}^{n-2} i \\ &= (n-1)^2 - \frac{(n-2)(n-1)}{2} = \frac{n(n-1)}{2} \in O(n^2) \end{aligned}$$

Example: Matrix Multiplication

```
function MATRIXMULT( $A[\cdot, \cdot], B[\cdot, \cdot], n$ )
```

```
  for  $i \leftarrow 0$  to  $n - 1$  do
```

```
    for  $j \leftarrow 0$  to  $n - 1$  do
```

```
       $C[i, j] \leftarrow 0$ 
```

```
      for  $k \leftarrow 0$  to  $n - 1$  do
```

```
         $C[i, j] \leftarrow C[i, j] + A[i, k] \cdot B[k, j]$ 
```

```
  return  $C$ 
```

▷ For $n \times n$ matrices

We count the **number of multiplications** executed:

$$M(n) = \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} \sum_{k=0}^{n-1} 1$$



Research Progress on Matrix Multiplication

- Volker Strassen (1969): $O(n^{2.8074})$;
- Josh Alman and Virginia Vassilevska Williams (2021): $O(n^{2.3728596})$;
- Duan, Wu and Zhou (Oct 2022): $O(n^{2.37188})$;
- The **optimal** bound for matrix multiplication is still **unknown** yet; and this is a **major open question** in theoretical computer science.

Analysing Recursive Algorithms

Let us start with a simple example:

```
function F( $n$ )  
  if  $n = 0$  then return 1  
  else return F( $n - 1$ ) ·  $n$ 
```

The basic operation here is the multiplication.

We express the cost recursively as well:

$$\begin{aligned}M(0) &= 0 \\M(n) &= M(n-1) + 1 \quad \text{for } n > 0\end{aligned}$$

To find a **closed form**, that is, one without recursion, we usually try “telescoping”, or “backward substitutions” in the recursive part.

The recursive equation was:

$$M(n) = M(n - 1) + 1 \quad (\text{for } n > 0)$$

Use the fact $M(n - 1) = M(n - 2) + 1$ to expand the right-hand side:

$$M(n) = [M(n - 2) + 1] + 1 = M(n - 2) + 2$$

and keep going:

$$\dots = [M(n - 3) + 1] + 2 = M(n - 3) + 3 = \dots = M(n - n) + n = n$$

where we used the base case $M(0) = 0$ to finish.

A Second Example: Binary Search in Sorted Array

```
function BINSEARCH( $A[\cdot]$ ,  $lo$ ,  $hi$ ,  $key$ )  
  if  $lo > hi$  then return  $-1$   
   $mid \leftarrow lo + (hi - lo)/2$   
  if  $A[mid] = key$  then return  $mid$   
  else  
    if  $A[mid] > key$  then  
      return BINSEARCH( $A$ ,  $lo$ ,  $mid - 1$ ,  $key$ )  
    else return BINSEARCH( $A$ ,  $mid + 1$ ,  $hi$ ,  $key$ )
```

The basic operation is the key comparison.

The cost, recursively, in the worst case:

$$\begin{aligned}C(1) &= 1 \\C(n) &= C(n/2) + 1 \quad \text{for } n > 0\end{aligned}$$

Telescoping

A **smoothness rule** allows us to assume that n is a power of 2. Why?

The recursive equation was:

$$C(n) = C(n/2) + 1 \text{ (for } n > 0\text{)}$$

Use the fact $C(n/2) = C(n/4) + 1$ to expand, and keep going:

$$\begin{aligned} C(n) &= C(n/2) + 1 \\ &= [C(n/4) + 1] + 1 \\ &= [[C(n/8) + 1] + 1] + 1 \\ &\vdots \\ &= \underbrace{[[\dots [[C(1) + 1] + 1] + \dots + 1] + 1]}_{1 + \log_2 n \text{ times}} \end{aligned}$$

Hence, $C(n) \in O(\log n)$.

Logarithmic Functions Have Same Rate of Growth

In big- O notation, we can just write “log” for any logarithmic function **with constant base**, no matter what its base is.

Asymptotically, for any positive *constants* $a \neq 1$ and $b \neq 1$, all logarithmic behaviour is the same, since

$$\log_a x = (\log_a b)(\log_b x)$$

So, for example, if $\ln n$ is the natural logarithm then

$$\begin{aligned}\log_2 n &\in O(\ln n) \\ \ln n &\in O(\log_2 n)\end{aligned}$$

Also note that since $\log(n^c) = c \cdot \log n$, we have, for all constants c ,

$$\log(n^c) \in O(\log n)$$

Back to Euclid

In Lecture 1 we considered Euclid's algorithm; here it is as a function written in pseudo-code (recursive version on the right):

```
function EUCLID( $m, n$ )  
  while  $n \neq 0$  do  
     $r \leftarrow m \bmod n$   
     $m \leftarrow n$   
     $n \leftarrow r$   
  return  $m$ 
```

```
function EUCLID( $m, n$ )  
  if  $n = 0$  then  
    return  $m$   
  return EUCLID( $n, m \bmod n$ )
```

We claim the running time is bounded by $O(\log(m + n))$.

To see this, assume $1 < n < m$.

After two iterations, m 's value has changed to $m \bmod n$.

Since $1 < n < m$, we have $m \bmod n < m/2$.

Question: Can you prove this?

Therefore, after every two iterations, the value of m decreases by a factor of 2.

The analysis is analogous to that of the binary search.

Summarising Reasoning with Big-Oh

$$O(f(n)) + O(g(n)) = O(\max\{f(n), g(n)\})$$

$$c \cdot O(f(n)) = O(f(n))$$

$$O(f(n)) \cdot O(g(n)) = O(f(n) \cdot g(n)).$$

The first equation justifies throwing smaller summands away.

The second says that constants can be thrown away too.

The third may be used with some nested loops. Suppose we have a loop which is executed $O(f(n))$ times, and each execution takes time $O(g(n))$. Then the execution of the loop takes time $O(f(n) \cdot g(n))$.

Some Useful Formulas

From Stirling's formula:

$$n! \in O(n^{n+\frac{1}{2}})$$

Some useful sums:

$$\sum_{i=0}^n i^2 = \frac{n}{3}(n + \frac{1}{2})(n + 1)$$

$$\sum_{i=0}^n (2i + 1) = (n + 1)^2$$

$$\sum_{i=1}^n 1/i \in O(\log n)$$

See also Cormen's Appendix A or Levitin's Appendix A.

Levitin's Appendix B is a tutorial on recurrence relations.

The Road Ahead

You will become much more familiar with asymptotic analysis as we use it on algorithms that we meet.

Next week we begin the study of algorithms by looking at **brute force** approaches.